

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 3
Student Name: K. Sirisha
Roll No.: A24126552260
Section: B

1. TITLE

Student Profile Manager Using JavaScript Class and DOM

2. AIM

To develop a Student Profile Manager web application that uses a JavaScript class to model student data and the DOM to dynamically add, display and delete student profiles.

3. OBJECTIVE

The objectives of this experiment are:

- To define a JavaScript class representing a Student object.
- To create a manager class that maintains a collection of objects.
- To capture form input and create new objects from it.
- To dynamically generate DOM elements to display each object.
- To attach event listeners for adding and deleting profiles.
- To update the DOM in real time whenever the underlying data changes.

4. THEORY

A JavaScript `class` is a template for creating objects that bundle related data (properties) and behaviour (methods) together. The `constructor()` method initializes an object's properties when it is created with the `new` keyword.

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. Methods such as `document.createElement()`, `element.appendChild()` and `element.innerHTML` are used to build and update page content dynamically, without reloading the page. Event listeners, attached using `addEventListener()`, let the page respond to user actions such as submitting a form or clicking a button.

In this application, a `Student` class stores each student's name, roll number, branch and email. A `StudentManager` class holds an array of `Student` objects and is responsible for adding new students, removing students, and rendering the current list to the DOM as a set of profile cards.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file with a form (Name, Roll Number, Branch, Email) and a container div for the profile list.
2. Define a `Student` class with a constructor that stores name, roll, branch and email.
3. Define a `StudentManager` class with an array of students and methods `addStudent()`, `deleteStudent()` and `render()`.
4. In `render()`, clear the container and use `createElement` / `innerHTML` to build a card for every student in the array.
5. Create a `StudentManager` instance and attach a `submit` event listener to the form.
6. On submit, prevent the default page reload, read the form values, create a new `Student` object and call `addStudent()`.
7. Inside `render()`, attach a `click` event listener to each card's Delete button that calls `deleteStudent()` with that card's index.
8. Update a counter element to show the total number of students whenever the list changes.
9. Save the file, open it in a browser, and add/delete a few student profiles to verify the behaviour.

6. PROGRAM

index.html

```
<!DOCTYPE html>
```



```

deleteStudent(index) {
  this.students.splice(index, 1);
  this.render();
}

render() {
  // Clear existing DOM content
  this.listElement.innerHTML = "";

  this.students.forEach((s, index) => {
    const card = document.createElement("div");
    card.className = "card";

    card.innerHTML = `
      <button class="del" data-index="${index}">Delete</button>
      <h3>${s.name}</h3>
      <p><b>Roll No:</b> ${s.roll}</p>
      <p><b>Branch:</b> ${s.branch}</p>
      <p><b>Email:</b> ${s.email}</p>
    `;

    this.listElement.appendChild(card);
  });

  this.countElement.textContent = `Total Students: ${this.students.length}`;

  // Attach delete event listeners after rendering
  this.listElement.querySelectorAll(".del").forEach(btn => {
    btn.addEventListener("click", (e) => {
      const idx = parseInt(e.target.getAttribute("data-index"));
      this.deleteStudent(idx);
    });
  });
}

const listEl = document.getElementById("studentList");
const countEl = document.getElementById("count");
const manager = new StudentManager(listEl, countEl);

// ---- Form submit event listener ----
document.getElementById("studentForm").addEventListener("submit", function (e) {
  e.preventDefault();

  const name = document.getElementById("name").value;
  const roll = document.getElementById("roll").value;
  const branch = document.getElementById("branch").value;
  const email = document.getElementById("email").value;

  const newStudent = new Student(name, roll, branch, email);
  manager.addStudent(newStudent);

  this.reset();
});

// ---- Pre-load sample data to demonstrate the DOM rendering ----
manager.addStudent(new Student("K. Sirisha", "A24126552260", "CSE", "sirisha@example.edu"));
manager.addStudent(new Student("K. Priya", "A24126552258", "ECE", "priya@example.edu"));
manager.addStudent(new Student("M. Arjun", "A24126552259", "MECH", "arjun@example.edu"));
</script>

</body>
</html>

```

7. CODE EXPLANATION

Code	Explanation
<code>class Student { constructor(...) }</code>	Defines the blueprint for a single student object with name, roll, branch and email.
<code>class StudentManager</code>	Manages the array of Student objects and keeps the DOM in sync with it.
<code>addStudent(student)</code>	Pushes a new Student object into the array and re-renders the list.
<code>deleteStudent(index)</code>	Removes a student at the given index using <code>splice()</code> and re-renders.
<code>render()</code>	Clears the container and rebuilds a card for every student currently in the array.
<code>document.createElement("div")</code>	Creates a new DOM element to represent a student card.
<code>form.addEventListener("submit", ...)</code>	Listens for form submission to create and add a new student.
<code>e.preventDefault()</code>	Stops the browser from reloading the page on form submit.
<code>btn.addEventListener("click", ...)</code>	Listens for clicks on each card's Delete button and removes that student.

8. TEST CASES AND EXECUTION DATA

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Load the page in browser	Sample student profiles are displayed as cards on load	3 sample cards displayed	Pass
TC-02	Fill the form with a new student's details and submit	A new card is added to the list and the form clears	New card added, form reset	Pass
TC-03	Submit the form with Name field empty	Browser should require the Name field	Browser blocked submission and asked to fill Name	Pass
TC-04	Click "Delete" on a student card	That card is removed and the count updates	Card removed, count updated correctly	Pass
TC-05	Add multiple students in sequence	"Total Students" counter increases correctly each time	Counter updated correctly for each addition	Pass

9. OUTPUT

Output 1: Initial Page with Pre-loaded Student Profiles

Student Profile Manager

Add Student Profile

Full Name

Roll Number

Branch

CSE

Email

Add Student

Total Students: 3

K. Sirisha

Roll No: A24126552260

Branch: CSE

Email: sirisha@example.edu

Delete

K. Priya

Roll No: A24126552258

Branch: ECE

Email: priya@example.edu

Delete

M. Arjun

Roll No: A24126552259

Branch: MECH

Email: arjun@example.edu

Delete

Rendered in browser — registration form and three student profile cards generated dynamically from the Student array via the render() method.

Output 2: Adding a New Student — Form Filled Before Submit

Student Profile Manager

Add Student Profile

Full Name

B. Rakesh

Roll Number

A24126552260

Branch

CIVIL

Email

rakesh@example.edu

Add Student

Total Students: 3

Form filled with a new student's details, ready to be submitted; the submit event listener will create a Student object and call addStudent().

10. RESULT

Thus, a Student Profile Manager web application was successfully developed using a JavaScript class to model student data and DOM manipulation to dynamically add, display and delete student profiles. The output was verified against the expected results for all test cases.